Final Project EDA

Romy Gou

2025-03-07

Preparing Dataset

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(tools)
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(tidymodels)
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.2.0 --
## v broom        1.0.7     v rsample      1.2.1
## v dials        1.3.0     v tune         1.2.1
## v infer        1.0.7     v workflows    1.1.4
## v modeldata    1.4.0     v workflowsets 1.1.0
## v parsnip      1.2.1     v yardstick    1.3.1
## v recipes      1.1.0
## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x scales::discard()        masks purrr::discard()
## x dplyr::filter()          masks stats::filter()
## x recipes::fixed()         masks stringr::fixed()
## x dplyr::lag()             masks stats::lag()
## x caret::lift()            masks purrr::lift()
## x yardstick::precision()   masks caret::precision()
```

```
## x yardstick::recall()      masks caret::recall()
## x yardstick::sensitivity() masks caret::sensitivity()
## x yardstick::spec()        masks readr::spec()
## x yardstick::specificity() masks caret::specificity()
## x recipes::step()          masks stats::step()
## * Use suppressPackageStartupMessages() to eliminate package startup messages
```

```
library(vip)
```

```
##
## Attaching package: 'vip'
##
## The following object is masked from 'package:utils':
##
##     vi
```

```
spotify <- read.csv("/Users/romygou/Documents/winter 2025/stats 140xp/spotify.csv")
spotify <- spotify %>%
  select(acousticness, danceability, energy, duration_ms, instrumentalness, valence, tempo, liveness, l
  na.omit()
```

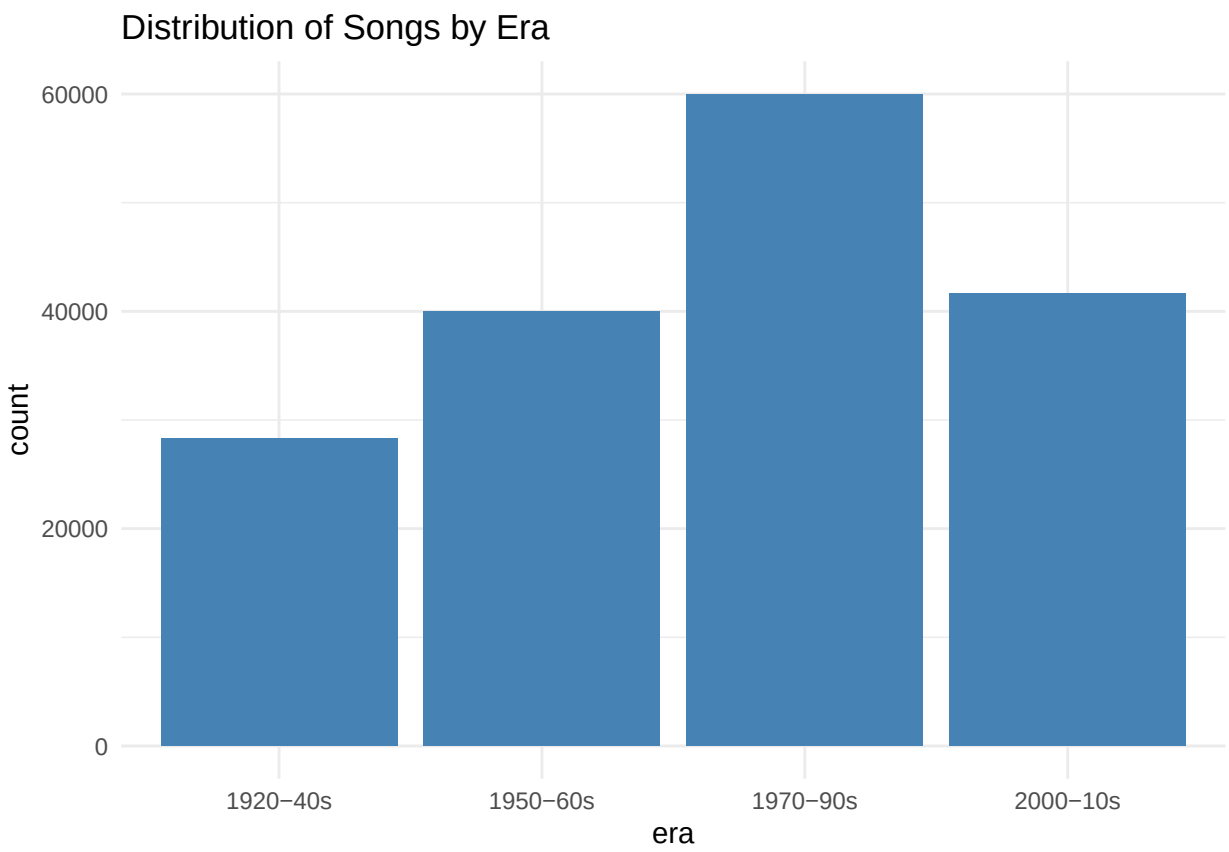
Instead of using 'year' as the target, we'll categorize each song into the era it was released. This will make it easier to create visualizations and a classification model. The classes of our new target variable, 'era', are: '1920-40s' (jazz, swing, big band), '1950-60s' (pop, doo-wop, rock and roll), '1970-90s' (disco, funk, dance, hip-hop), and '2000-10s' (pop, hip-hop).

```
spotify <- spotify %>%
  mutate(era = case_when(
    year >= 1920 & year <= 1949  ~ "1920-40s",
    year >= 1950 & year <= 1969  ~ "1950-60s",
    year >= 1970 & year <= 1999  ~ "1970-90s",
    year >= 2000 & year <= 2020  ~ "2000-10s")) %>%
  select(-year)
spotify$era <- as.factor(spotify$era)

# normalized data
spotify_norm <- spotify %>%
  mutate(across(where(is.numeric), ~ (. - min(.)) / (max(.) - min(.))))
```

Visualizations

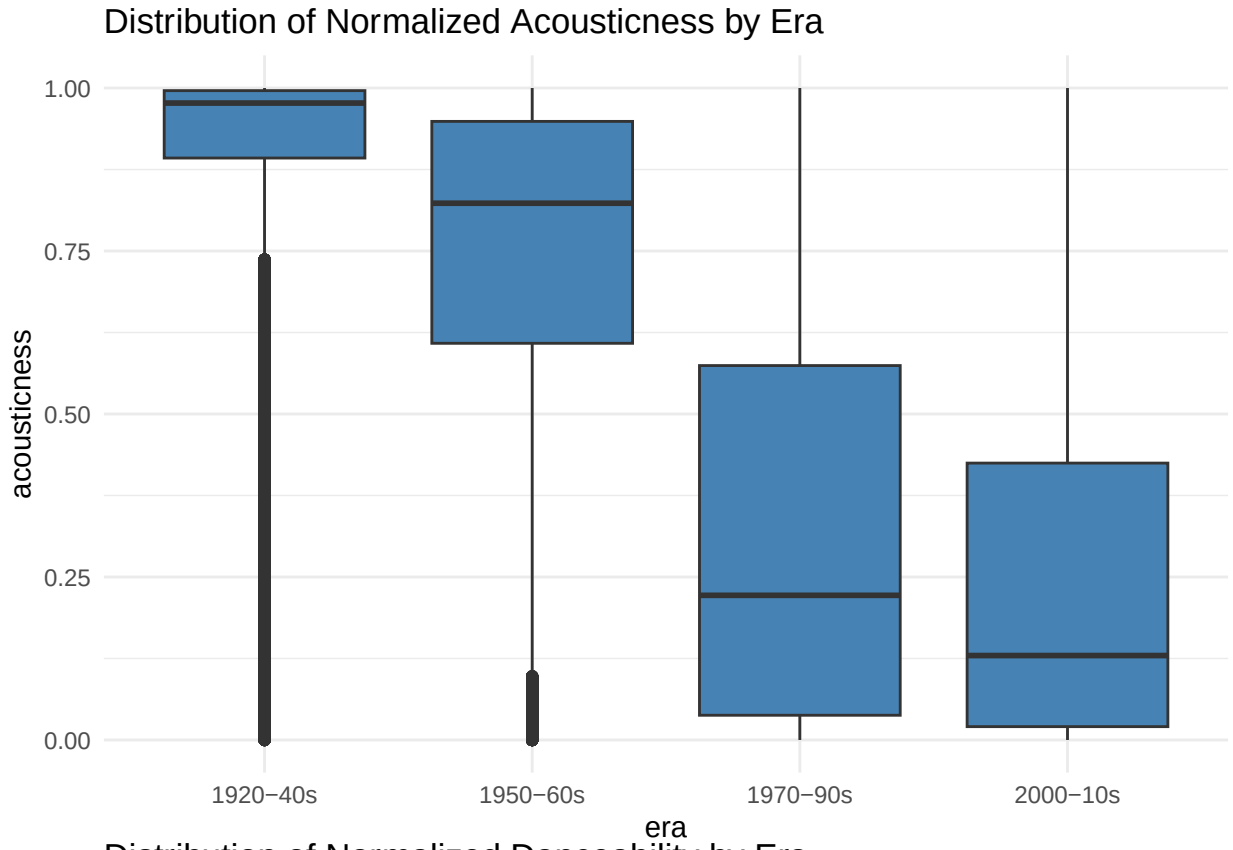
Distribution of classes within target variable:

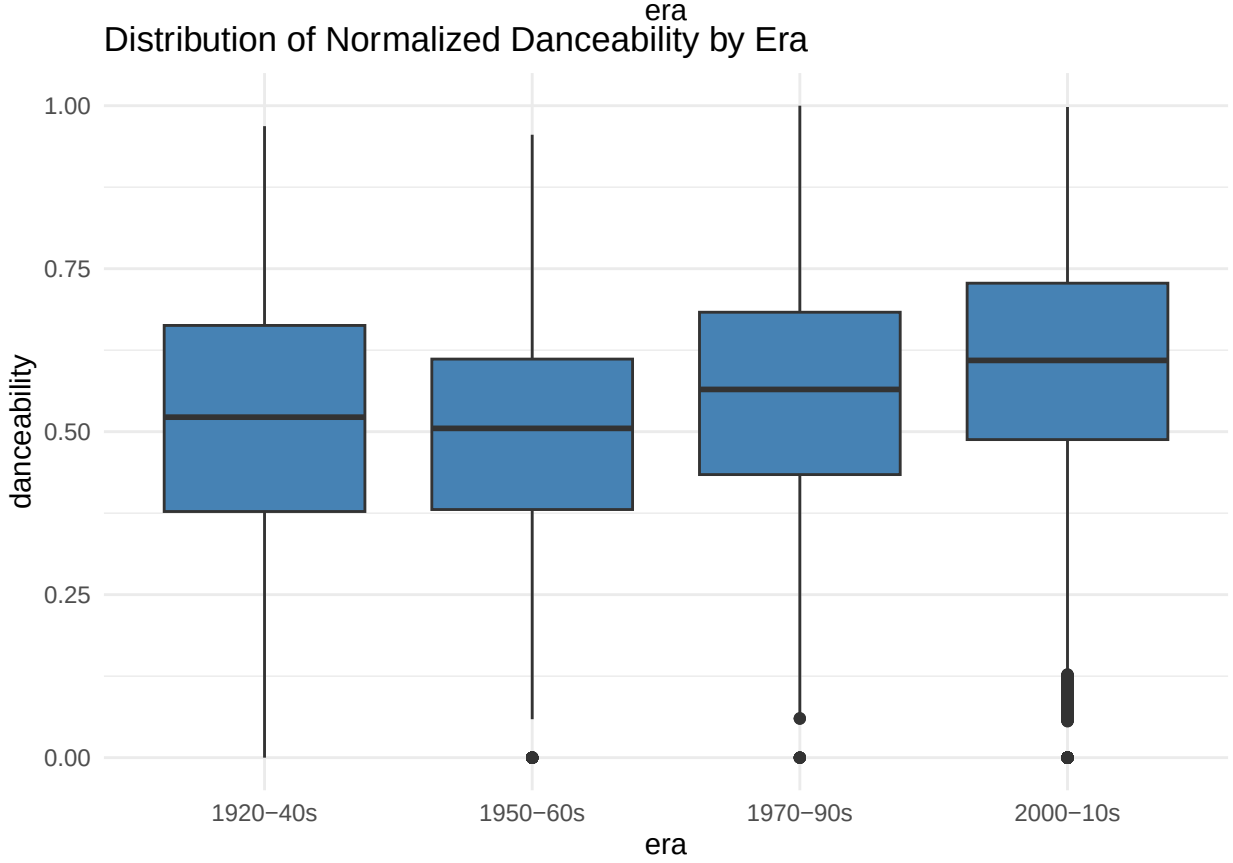
```
ggplot(spotify_norm) +
  aes(x = era) +
  geom_bar(fill = "#4682B4") +
  ggtitle(paste("Distribution of Songs by Era")) +
  theme_minimal()
```

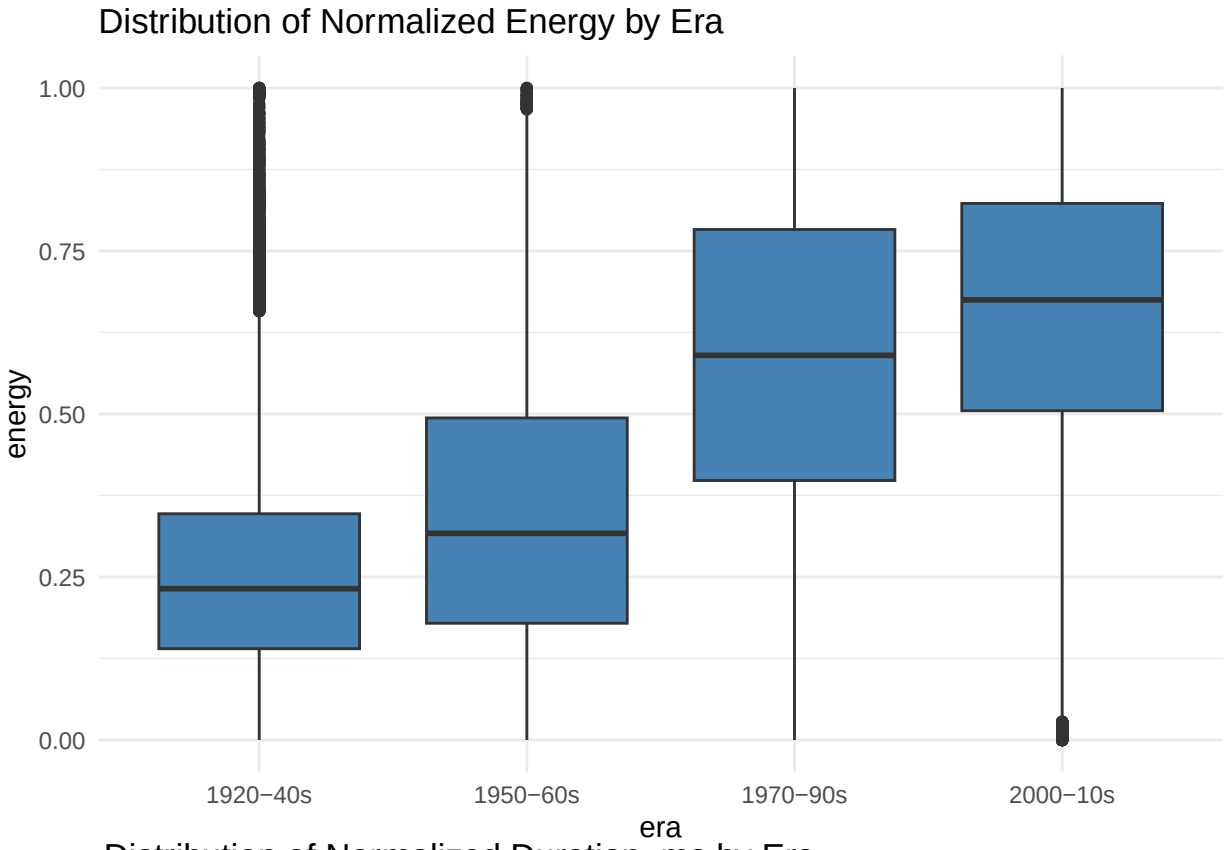


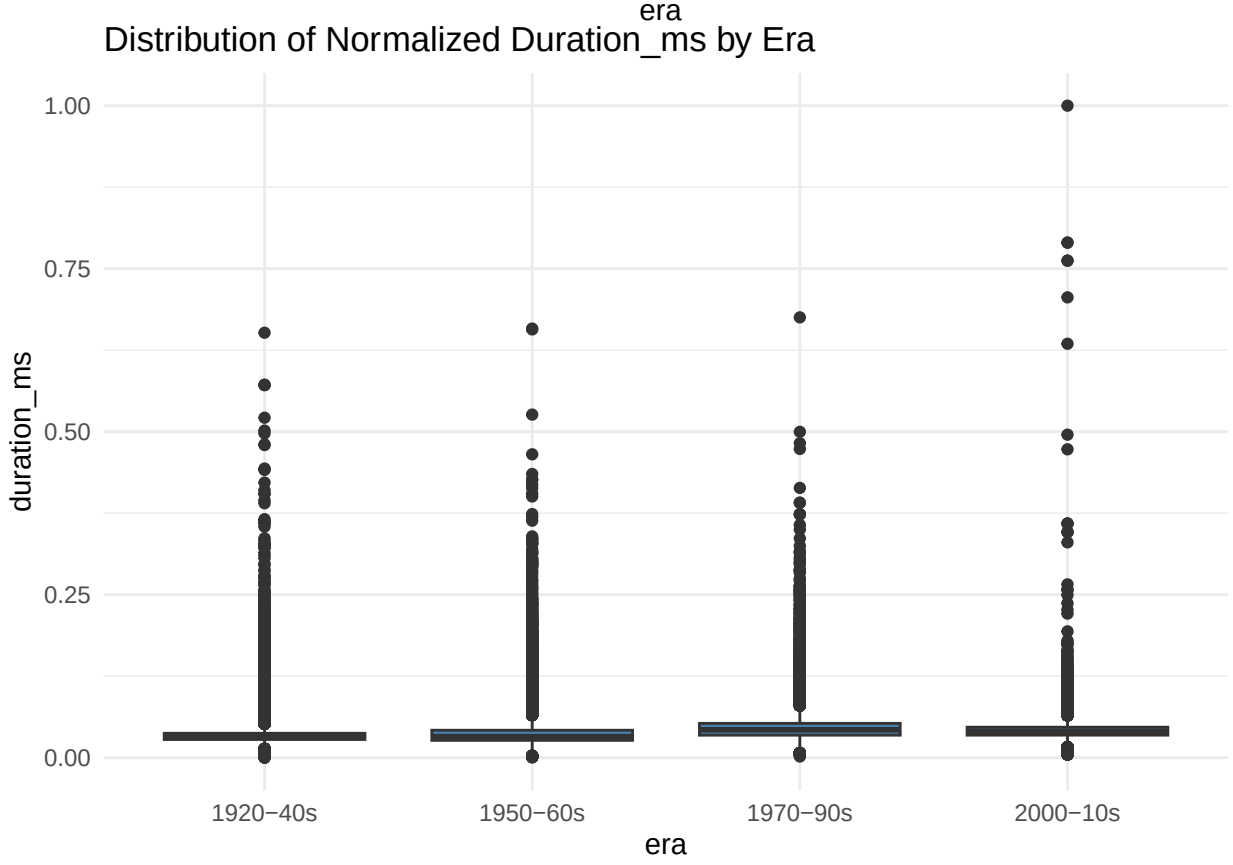
The classes are unbalanced, which means that we should either implement class weights or choose a classification model that handles imbalances well.

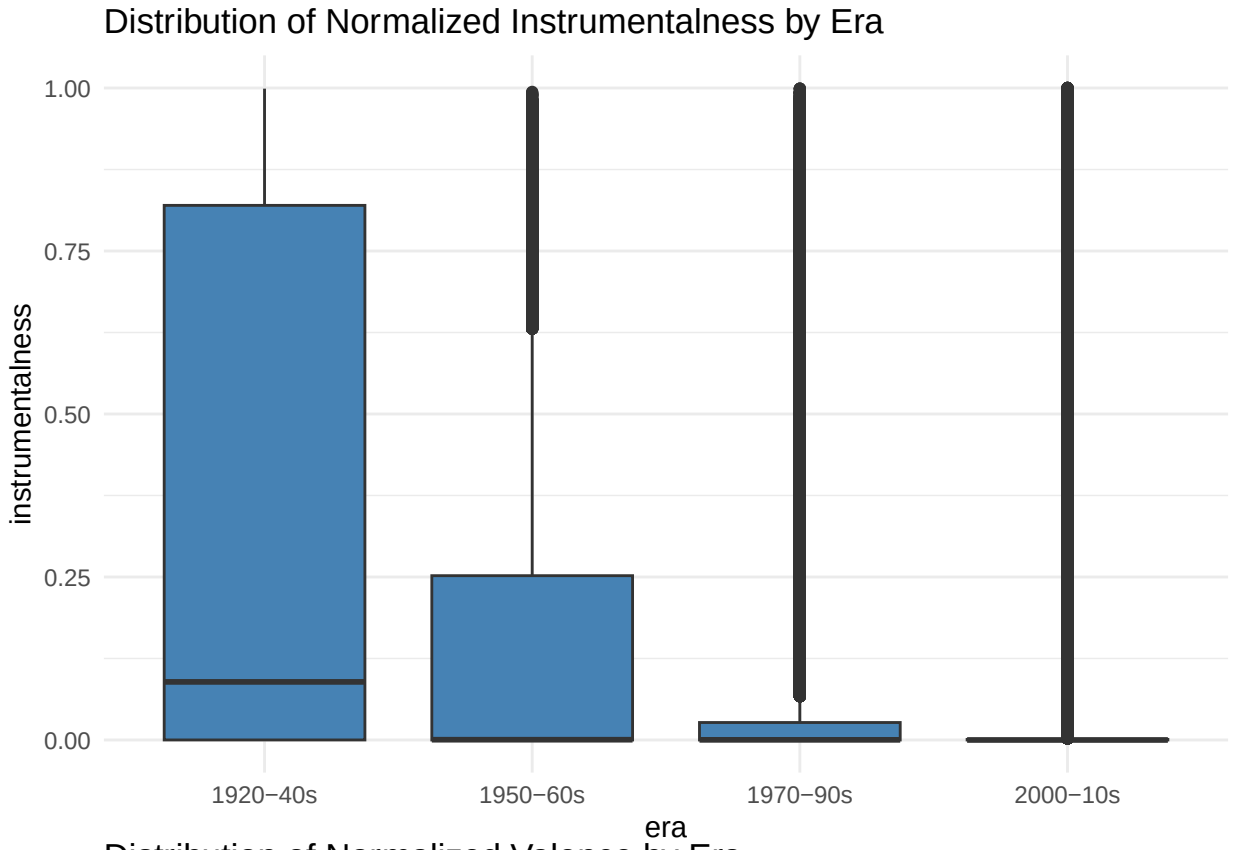
```
features <- c("acousticness", "danceability", "energy", "duration_ms", "instrumentalness", "valence", "

for (i in 1:11) {
  feature_plot <- ggplot(spotify_norm, aes(x = era, y = .data[[features[i]]])) +
    geom_boxplot(fill = "#4682B4") +
    theme_minimal() +
    ggtitle(paste("Distribution of Normalized", toTitleCase(features[i]), "by Era"))
  print(feature_plot)}
```

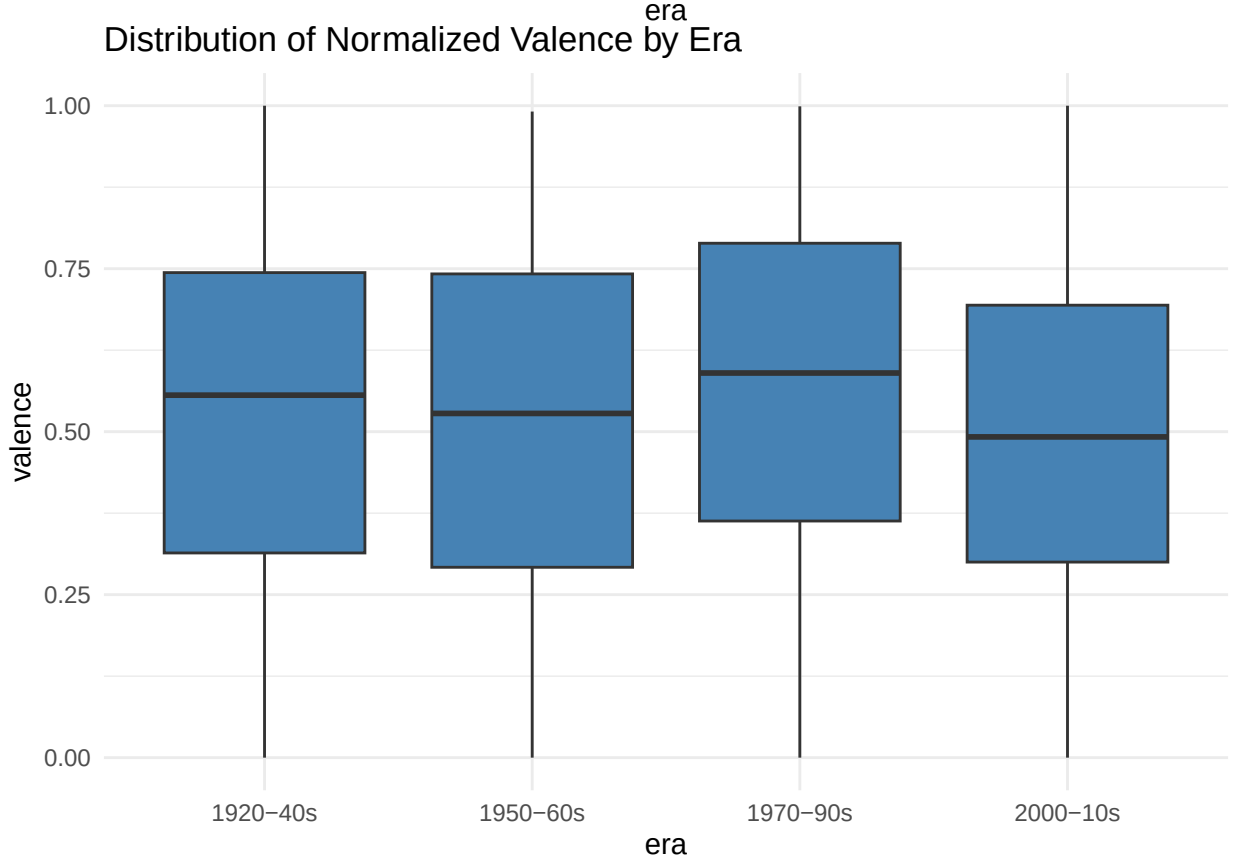

Distribution of Normalized Acousticness by Era
acousticness
1.00
0.75
0.50
0.25
0.00
1920−40s
1950−60s
1970−90s
2000−10s
era

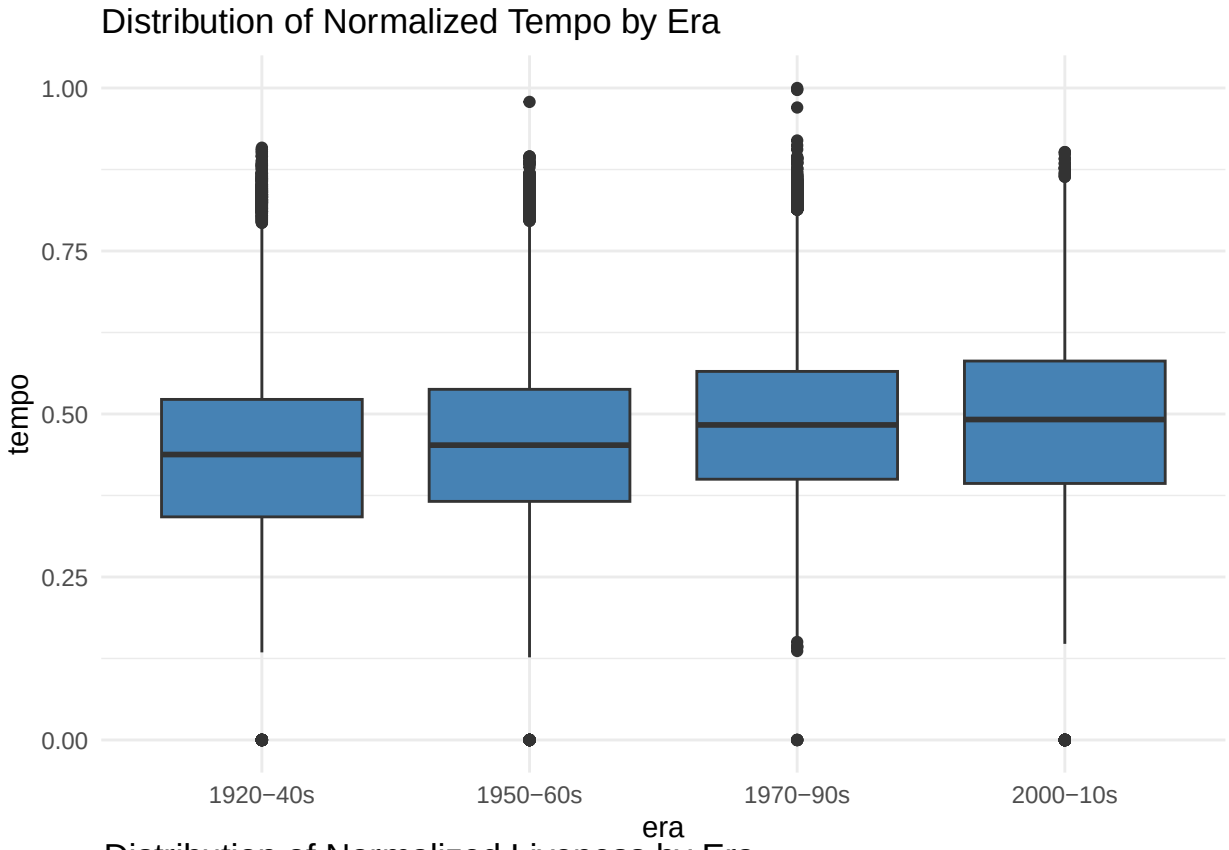

Distribution of Normalized Danceability by Era
danceability
1.00
0.75
0.50
0.25
0.00
1920−40s
1950−60s
1970−90s
2000−10s
era

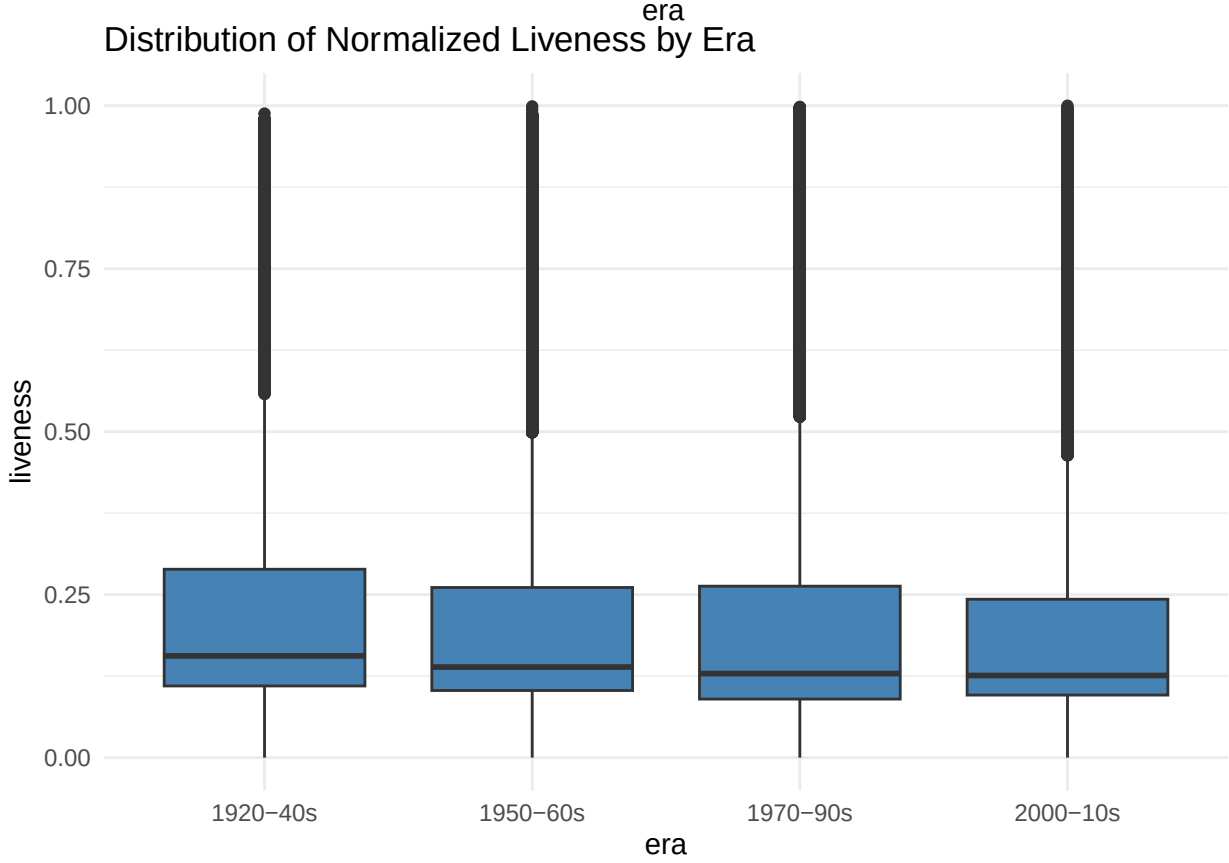

Distribution of Normalized Energy by Era
1.00
0.75
0.50
0.25
0.00
energy
1920−40s
1950−60s
1970−90s
2000−10s
era

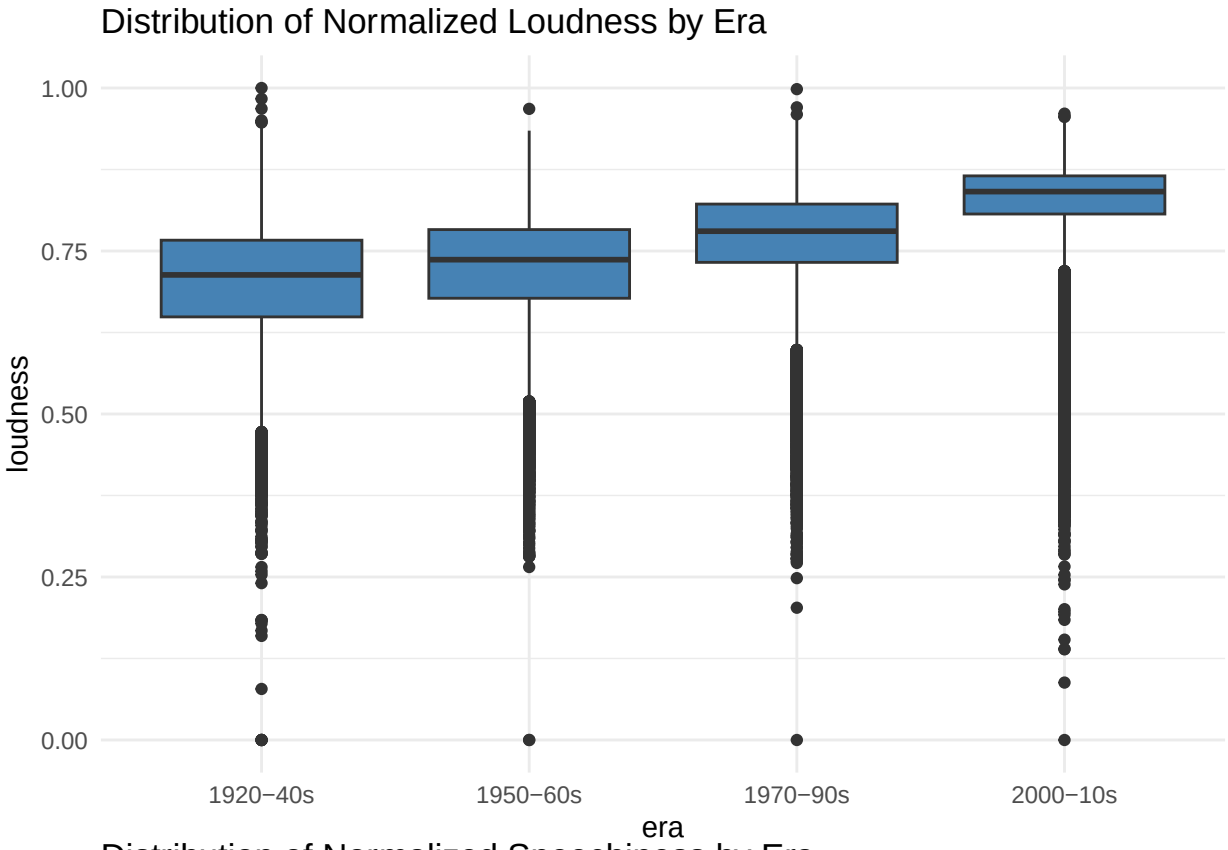

Distribution of Normalized Duration_ms by Era
1.00
0.75
0.50
0.25
0.00
duration_ms
1920−40s
1950−60s
1970−90s
2000−10s
era

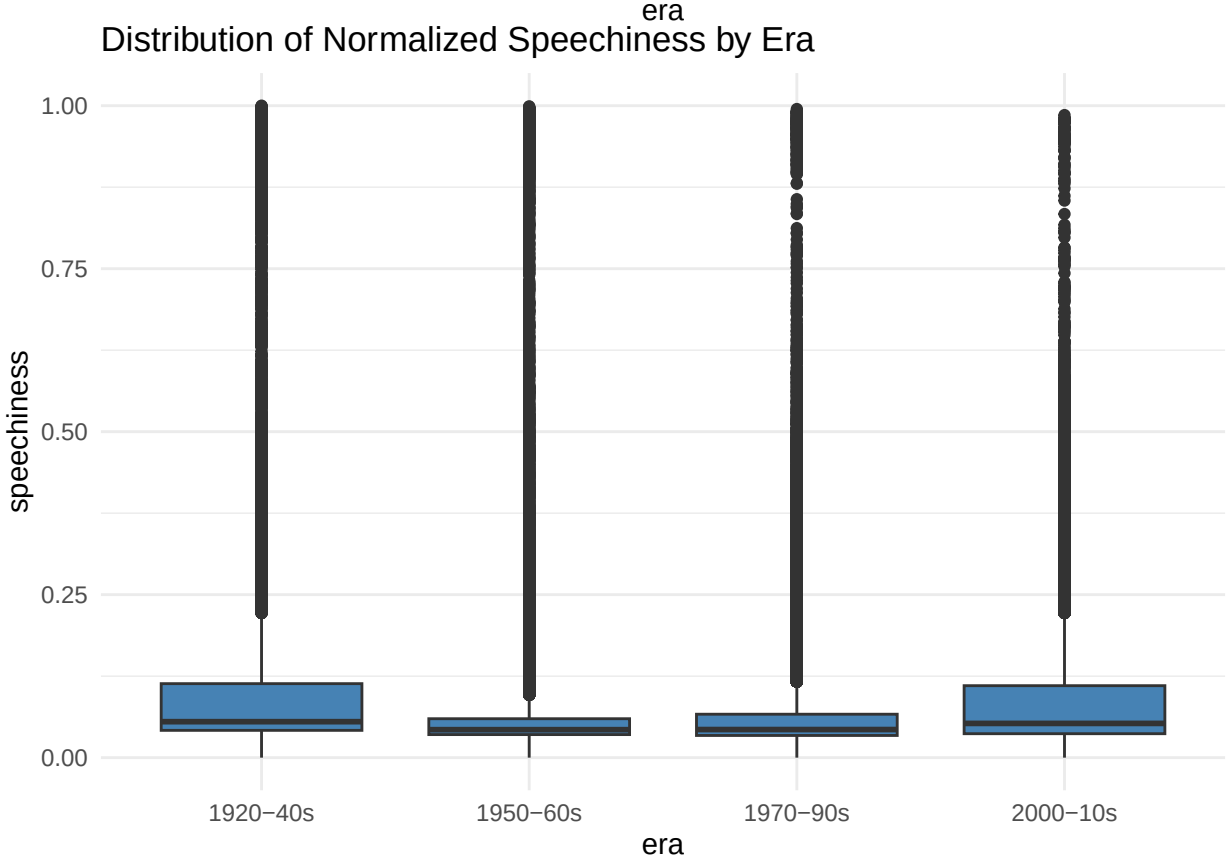

Distribution of Normalized Instrumentalness by Era
instrumentalness
1.00
0.75
0.50
0.25
0.00
1920−40s
1950−60s
1970−90s
2000−10s
era

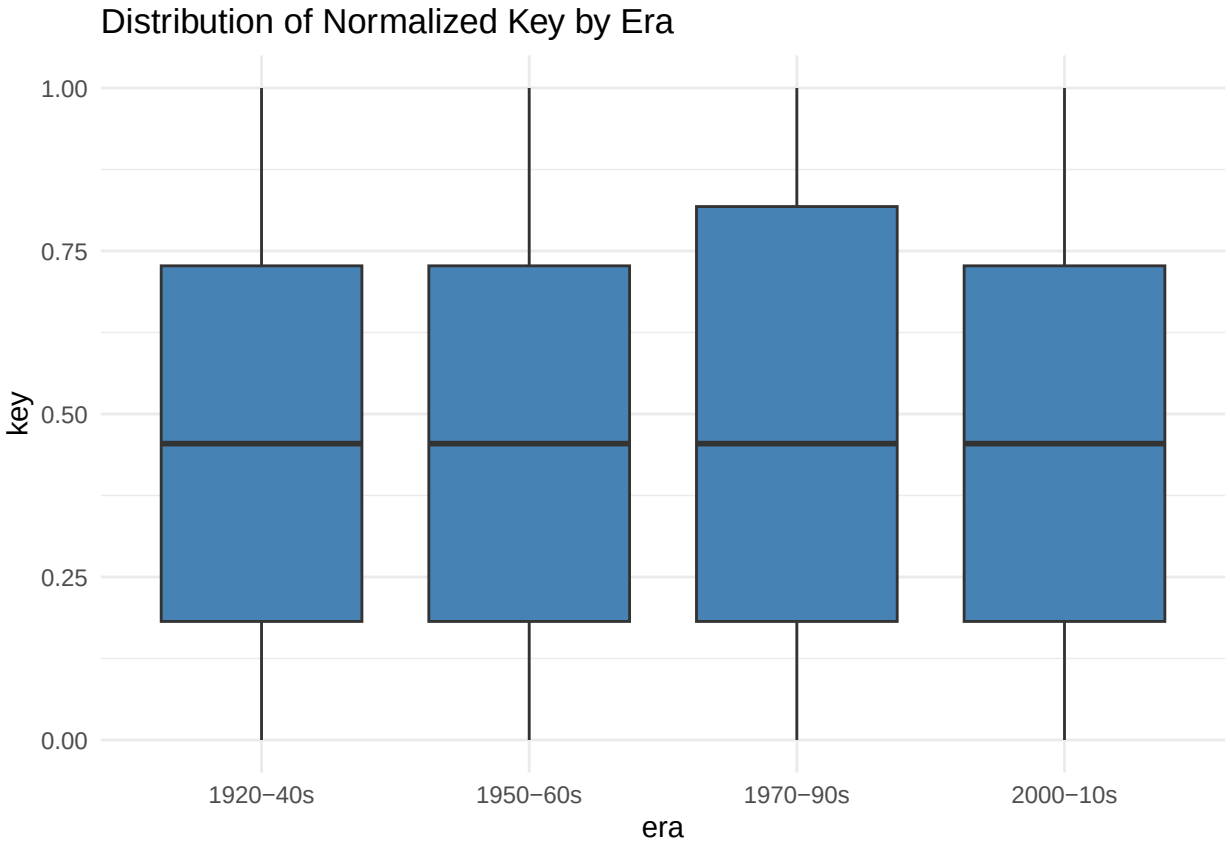

Distribution of Normalized Valence by Era
valence
1.00
0.75
0.50
0.25
0.00
1920−40s
1950−60s
1970−90s
2000−10s
era


Distribution of Normalized Tempo by Era
1.00
0.75
0.50
0.25
0.00
tempo
1920−40s
1950−60s
1970−90s
2000−10s
era


Distribution of Normalized Liveness by Era
1.00
0.75
0.50
0.25
0.00
liveness
1920−40s
1950−60s
1970−90s
2000−10s
era


Distribution of Normalized Loudness by Era
1.00
0.75
0.50
0.25
0.00
loudness
1920−40s
1950−60s
1970−90s
2000−10s
era


Distribution of Normalized Speechiness by Era
1.00
0.75
0.50
0.25
0.00
speechiness
1920−40s
1950−60s
1970−90s
2000−10s
era



Right away, we can see that there may be some features that will be less useful in classifying the release era of a song due to lack of variation between eras, such as 'key' and 'liveness'.

```
# table of standard deviation values
spotify %>%
  group_by(era) %>%
  summarise(across(where(is.numeric), sd))
```

```
## # A tibble: 4 x 12
##   era      acousticness danceability energy duration_ms instrumentalness valence
##   <fct>           <dbl>        <dbl>  <dbl>       <dbl>            <dbl>   <dbl>
## 1 1920-40s        0.242        0.178  0.168     151922.            0.391   0.266
## 2 1950-60s        0.268        0.161  0.218     138244.            0.337   0.270
## 3 1970-90s        0.311        0.173  0.246     107715.            0.253   0.259
## 4 2000-10s        0.292        0.173  0.225      90312.            0.220   0.250
## # i 5 more variables: tempo <dbl>, liveness <dbl>, loudness <dbl>,
## #   speechiness <dbl>, key <dbl>
```

Considering the class imbalance and the large size of the data set, we'll train a random forest model.

Random Forest Model

```
set.seed(123)
data_split <- initial_split(spotify, prop = 0.8)
train_data <- training(data_split)
test_data <- testing(data_split)
```

```r
rf_recipe <- recipe(era ~., data = train_data) %>%
  step_normalize(all_numeric())

rf_spec <- rand_forest() %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

rf_workflow <- workflow() %>%
  add_recipe(rf_recipe) %>%
  add_model(rf_spec)

rf_fit <- rf_workflow %>%
  fit(data = train_data)
```

```r
predictions <- predict(rf_fit, new_data = test_data) %>%
  pull(.pred_class)
metrics <- metric_set(accuracy)
performance <- test_data %>%
  mutate(predictions = predictions) %>%
  metrics(truth = era, estimate = predictions)
print(performance)
```

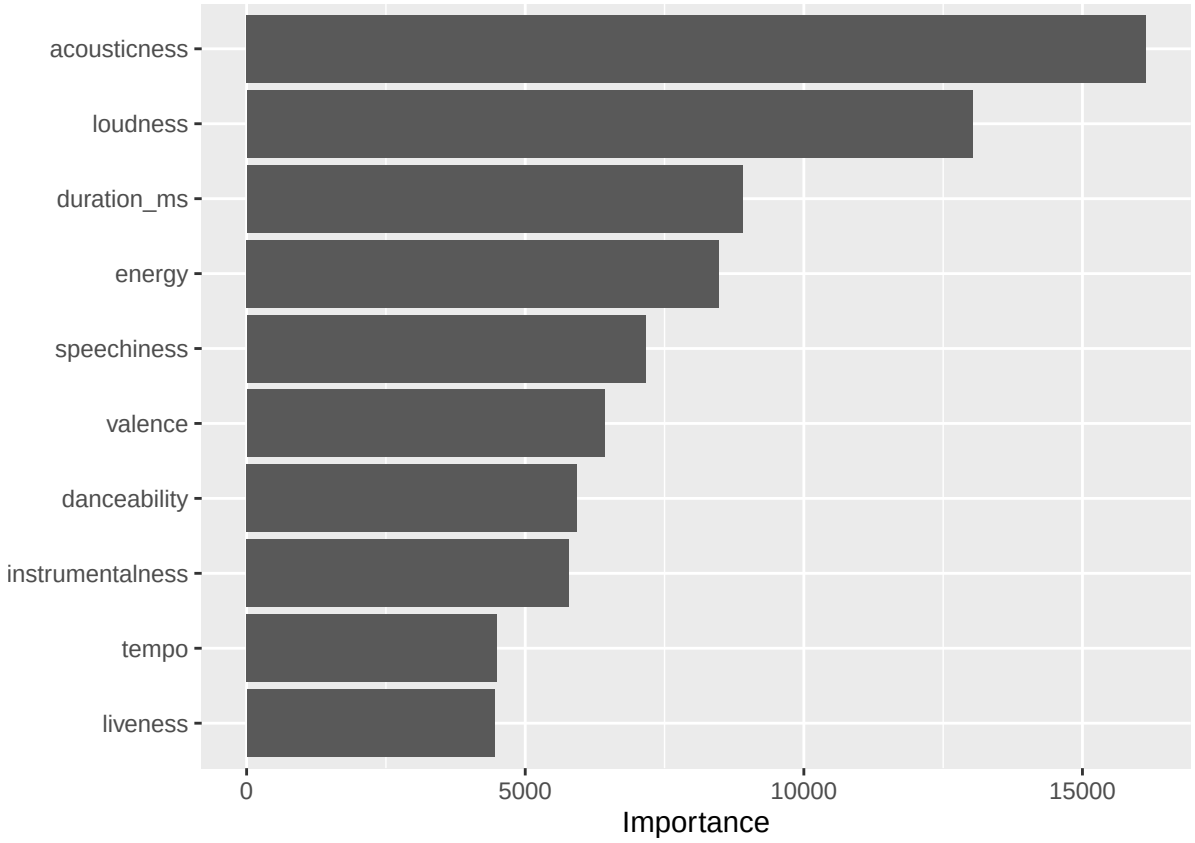
```
## # A tibble: 1 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy multiclass     0.702
```

```r
# confusion matrix
confusionMatrix(predictions, test_data$era)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction 1920-40s 1950-60s 1970-90s 2000-10s
##   1920-40s     4151     1128      198       97
##   1950-60s     1211     5299     1594      589
##   1970-90s      204     1413     8609     1902
##   2000-10s       62      216     1507     5802
##
## Overall Statistics
##
##                Accuracy : 0.7022
##                  95% CI : (0.6973, 0.707)
##     No Information Rate : 0.3504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.5931
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: 1920-40s Class: 1950-60s Class: 1970-90s
## Sensitivity                   0.7376          0.6578          0.7230
## Specificity                   0.9498          0.8691          0.8406
```

```
## Pos Pred Value                0.7447          0.6096          0.7098
## Neg Pred Value                0.9480          0.8910          0.8490
## Prevalence                    0.1656          0.2371          0.3504
## Detection Rate                0.1222          0.1559          0.2533
## Detection Prevalence          0.1640          0.2558          0.3569
## Balanced Accuracy             0.8437          0.7634          0.7818
##                       Class: 2000-10s
## Sensitivity                    0.6915
## Specificity                    0.9303
## Pos Pred Value                 0.7647
## Neg Pred Value                 0.9020
## Prevalence                     0.2469
## Detection Rate                 0.1707
## Detection Prevalence           0.2233
## Balanced Accuracy              0.8109
```

```
vip(rf_fit)
```



Research Questions:

Can we predict the era a song was released based on different musical metrics, such as loudness, tempo, danceability, and energy?

Answer: Yes, we can produce a random forest model with 70.2% accuracy.

Which musical metrics are most indicative of the era released?

Answer: Acousticness and loudness are most indicative of era released.

For which eras, if any, are musical metrics most similar? What does this say about the recycling of musical trends or the impact of cultural context on music? (Comparing metrics across eras)

Answer: 1920-40s music is most often misclassified as 1950-60s music, 1950-60s music is most often misclassified as 1970-90s music, 1970-90s music is most often misclassified as 1950-60s music, and 2000-10s music is most often misclassified as 1970-90s music.

Does the uniformity of the musical metrics of same-era songs change throughout time? In which era are the songs most uniform? In which era are the songs most diverse? (Comparing metrics within an era)

Answer: The uniformity of musical metrics doesn't change significantly throughout time. However, by looking at the table of standard deviation values, we can see which eras had lower or higher standard deviations for certain musical metrics.